

Deep Learning — Manual Numerical Practice Problems

How to use this set: Every problem gives you all the numbers you need — nothing is left for you to guess. Work each problem by hand with a calculator. Solutions are **not** provided automatically. When you're ready, say:

▮ "Solve Problem X step by step."

...and I will show every arithmetic step, no skipping.

Problems are numbered sequentially (Problem 1, Problem 2, ...) across the whole set, grouped into the 32 topic sections below, in order of increasing difficulty.

Section 1 — Scalar Neuron

Problem 1: Single Scalar Neuron — Forward Pass

Given

```
Input: x = 3
Weight: w = 2
Bias: b = 1
Activation: identity (no activation, i.e. a = z)
Target: y = 8
```

Task

1. Calculate the weighted sum $z = wx + b$.
 2. Since activation is identity, state the prediction $\hat{y}$.
 3. Calculate the squared error loss $L = (\hat{y} - y)^2$.
-

Problem 2: Single Scalar Neuron — Full Update (Gradient Descent)

Given

```
Input: x = 3
Weight: w = 2
Bias: b = 1
Activation: identity
Target: y = 8
Loss: L = ( $\hat{y}$  - y)2
Learning Rate:  $\eta$  = 0.01
```

Task

1. Calculate z and $\hat{y}$.
 2. Calculate the loss L .
 3. Calculate $\partial L / \partial \hat{y}$.
 4. Calculate $\partial L / \partial w$ and $\partial L / \partial b$ (chain rule, using $\partial \hat{y} / \partial w = x$ and $\partial \hat{y} / \partial b = 1$).
 5. Update w and b using gradient descent: $w_{\text{new}} = w - \eta \nabla w$, $b_{\text{new}} = b - \eta \nabla b$.
 6. Recompute $\hat{y}$ with the new parameters and confirm the loss decreased.
-

Section 2 — Vector Neuron

Problem 3: Vector Neuron — Forward Pass and Loss

Given

```
Input: x = [0.5, 0.3]
Weight: w = [3, 3]
Bias: b = 0.1
Activation: identity
Target: y = 2
```

Task

1. Calculate $z = \Sigma(w_i x_i) + b$.
 2. State the prediction $\hat{y}$.
 3. Calculate the squared error loss $L = (\hat{y} - y)^2$.
-

Problem 4: Vector Neuron — Gradients and Update

Given

```
Input: x = [0.5, 0.3]
Weight: w = [3, 3]
Bias: b = 0.1
Activation: identity
Target: y = 2
Loss: L = ( $\hat{y}$  - y)2
Learning Rate:  $\eta$  = 0.1
```

Task

1. Calculate z and $\hat{y}$.
 2. Calculate the loss.
 3. Calculate $\partial L / \partial \hat{y}$ and $\partial L / \partial z$.
 4. Calculate the gradient vector $\partial L / \partial w = [\partial L / \partial w_1, \partial L / \partial w_2]$ and $\partial L / \partial b$.
 5. Update w_1 , w_2 , and b.
 6. Recompute $\hat{y}$ and confirm the loss decreased.
-

Section 3 — The Perceptron

Problem 5: Perceptron Learning Rule — Single Update

Given

```
Input: x = [0.5, 0.3]
Weight: w = [3, 3]
Bias: b = 0.1
Target (class label): y = 1
Activation: step function, f(z) = 1 if z ≥ 0 else 0
Learning Rate:  $\eta$  = 0.01
Perceptron rule:  $w_{\text{new}} = w + \eta(y - \hat{y})x$  ,  $b_{\text{new}} = b + \eta(y - \hat{y})$ 
```

Task

1. Calculate z.
 2. Apply the step activation to get $\hat{y}$.
 3. Calculate the error (y - $\hat{y}$).
 4. Update w_1 , w_2 , and b using the perceptron rule.
-

Problem 6: Perceptron — Binary Classification Over Multiple Points (by hand)

Given

Dataset (2D points, AND-like problem):

$$\mathbf{x}^{(1)} = [0, 0], y^{(1)} = 0$$

$$\mathbf{x}^{(2)} = [0, 1], y^{(2)} = 0$$

$$\mathbf{x}^{(3)} = [1, 0], y^{(3)} = 0$$

$$\mathbf{x}^{(4)} = [1, 1], y^{(4)} = 1$$

Initial weight: $\mathbf{w} = [0.1, 0.1]$

Initial bias: $b = -0.2$

Activation: step function, $f(z) = 1$ if $z \geq 0$ else 0

Learning Rate: $\eta = 0.1$

Task

1. Process the four points **in order, one at a time** (this is the perceptron/SGD-style update): for each point compute z , $\hat{y}$, the error, and update $\mathbf{w}$ and b immediately before moving to the next point.
 2. After one full pass (epoch) through all 4 points, report the final $\mathbf{w}$ and b .
 3. State whether all 4 points are now classified correctly with the final weights.
-

Section 4 — Activation Functions

Problem 7: ReLU — Forward and Derivative

Given

$$z = -2$$

Task: Calculate $\text{ReLU}(z)$ and $\text{ReLU}'(z)$.

Problem 8: ReLU — Gradient Flow

Given

$$z = 4$$

$$\text{Upstream gradient } (\partial L / \partial a) = 0.5$$

Task: Calculate $a = \text{ReLU}(z)$, then $\partial L / \partial z$ using the chain rule $\partial L / \partial z = \partial L / \partial a \cdot \text{ReLU}'(z)$.

Problem 9: Leaky ReLU — Forward, Derivative, Gradient

Given

```
z = -3
α (leak coefficient) = 0.01
Upstream gradient (∂L/∂a) = 2
```

Task

1. Calculate LeakyReLU(z).
 2. Calculate LeakyReLU'(z).
 3. Calculate $\partial L / \partial z$.
-

Problem 10: ELU — Forward, Derivative, Parameter Update

Given

```
z = -1
α = 1.0
Upstream gradient (∂L/∂a) = 1.5
```

Task

1. Calculate $\text{ELU}(z) = \alpha(e^z - 1)$ for $z < 0$. Use $e = 2.71828$.
 2. Calculate $\text{ELU}'(z)$ for $z < 0$, which equals $\text{ELU}(z) + \alpha$.
 3. Calculate $\partial L / \partial z$.
-

Problem 11: SELU — Forward and Derivative

Given

```
z = -0.5
λ = 1.0507
α = 1.6733
```

Task

1. Calculate $\text{SELU}(z) = \lambda \cdot \alpha (e^z - 1)$ for $z < 0$.
 2. Calculate $\text{SELU}'(z) = \lambda \cdot \alpha \cdot e^z$ for $z < 0$.
-

Problem 12: Sigmoid — Forward, Derivative, Gradient Update

Given

$z = 0.8$
Upstream gradient ($\partial L / \partial a$) = 0.3
Learning rate is not needed here – this problem only tests forward/backward, not a parameter update.

Task

1. Calculate $\sigma(z) = 1 / (1 + e^{-z})$. Use $e = 2.71828$.
 2. Calculate $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
 3. Calculate $\partial L / \partial z$.
-

Problem 13: Tanh — Forward, Derivative, Gradient

Given

$z = 0.5$
Upstream gradient ($\partial L / \partial a$) = -0.4

Task

1. Calculate $\tanh(z)$ using $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$. Use $e = 2.71828$.
2. Calculate $\tanh'(z) = 1 - \tanh(z)^2$.
3. Calculate $\partial L / \partial z$.

Where each activation sits in the graph: for a layer, the activation is always applied *after* the linear step: $z = wx + b \rightarrow a = f(z)$. During backprop, gradients flow *backward through the activation first* (multiplying by $f'(z)$) before reaching the weights: $\partial L / \partial z = \partial L / \partial a \cdot f'(z)$, then $\partial L / \partial W = \partial L / \partial z \cdot x^T$.

Section 5 — Single-Layer Forward Propagation

Problem 14: Single-Layer Network — Full Forward Pass

Given

```
Input: x = [1, 2]
Weight: w = [0.5, -0.5]
Bias: b = 0.2
Activation: sigmoid
Target: y = 1
Loss: Binary Cross-Entropy, L = -[y log(ŷ) + (1-y) log(1-ŷ)]
```

Task

1. Calculate $z = w \cdot x + b$.
 2. Calculate $\hat{y} = \sigma(z)$.
 3. Calculate the loss L.
-

Section 6 — MLP Forward Propagation

Problem 15: Two-Layer MLP — Full Forward Pass

Given

```
Input: X = [1, 2]
Layer 1 weights: W1 = [[0.1, 0.2],
                        [0.3, 0.4]]      (rows = neurons, columns = inputs)
Layer 1 bias: b1 = [0.1, 0.1]
Layer 1 activation: ReLU

Layer 2 (output) weights: W2 = [0.5, -0.5]  (1 output neuron, 2 inputs)
Layer 2 bias: b2 = 0.2
Layer 2 activation: sigmoid

Target: y = 1
Loss: Binary Cross-Entropy
```

Task

1. Calculate $Z1 = W1 \cdot X + b1$ (vector of 2 values).
2. Calculate $A1 = \text{ReLU}(Z1)$.

3. Calculate $Z2 = W2 \cdot A1 + b2$ (scalar).
 4. Calculate $A2 = \sigma(Z2)$ — this is $\hat{y}$.
 5. Calculate the loss L .
-

Section 7 — Backpropagation

Problem 16: Backprop Through One Hidden Layer

Given

```
Input: X = [1, 2]
W1 = [[0.1, 0.2],
      [0.3, 0.4]]
b1 = [0.1, 0.1]
Activation 1: ReLU

W2 = [0.5, -0.5]
b2 = 0.2
Activation 2: sigmoid

Target: y = 1
Loss: Binary Cross-Entropy
Learning Rate:  $\eta = 0.1$ 
```

(This continues directly from Problem 15's forward pass — you'll need $Z1$, $A1$, $Z2$, $A2$ from there.)

Task

1. Calculate $\partial L / \partial \hat{y}$ for BCE: $\partial L / \partial \hat{y} = -(y/\hat{y}) + (1-y)/(1-\hat{y})$.
 2. Calculate $\partial L / \partial Z2$ using the sigmoid+BCE shortcut: $\partial L / \partial Z2 = \hat{y} - y$.
 3. Calculate $\partial L / \partial W2 = \partial L / \partial Z2 \cdot A1^T$, and $\partial L / \partial b2 = \partial L / \partial Z2$.
 4. Calculate $\partial L / \partial A1 = \partial L / \partial Z2 \cdot W2$.
 5. Calculate $\partial L / \partial Z1 = \partial L / \partial A1 \odot \text{ReLU}'(Z1)$ (elementwise).
 6. Calculate $\partial L / \partial W1 = \partial L / \partial Z1 \cdot X^T$, and $\partial L / \partial b1 = \partial L / \partial Z1$.
 7. Update $W1$, $b1$, $W2$, $b2$ using gradient descent with $\eta = 0.1$.
-

Problem 17: Backprop Through Two Hidden Layers

Given

```
Input: X = [1, -1]

Layer 1: W1 = [[0.2, 0.1],
               [0.4, -0.2]]
        b1 = [0.0, 0.1]
        Activation: tanh

Layer 2: W2 = [[0.3, -0.1],
               [0.2, 0.5]]
        b2 = [0.1, -0.1]
        Activation: ReLU

Output Layer: W3 = [0.5, -0.3]
               b3 = 0.05
               Activation: sigmoid

Target: y = 0
Loss: Binary Cross-Entropy
Learning Rate:  $\eta = 0.05$ 
```

Task

1. Forward pass: compute Z_1 , A_1 , Z_2 , A_2 , Z_3 , $\hat{y}$, and the loss L .
2. Backward pass: compute $\partial L / \partial Z_3$, then $\partial L / \partial W_3$, $\partial L / \partial b_3$.
3. Backpropagate to layer 2: $\partial L / \partial A_2$, $\partial L / \partial Z_2$, $\partial L / \partial W_2$, $\partial L / \partial b_2$.
4. Backpropagate to layer 1: $\partial L / \partial A_1$, $\partial L / \partial Z_1$, $\partial L / \partial W_1$, $\partial L / \partial b_1$.
5. Update all parameters (W_1 , b_1 , W_2 , b_2 , W_3 , b_3).

Section 8 — Regression MLPs

Problem 18: Regression MLP with MSE

Given

```
Input: x = 2
W1 = [0.5, -0.3] (single hidden layer, 2 neurons, 1 input each)
b1 = [0.1, 0.2]
Activation 1: ReLU

W2 = [0.4, 0.6] (output neuron)
b2 = 0.0
Activation 2: identity (regression output)

Target: y = 3
Loss: MSE,  $L = (\hat{y} - y)^2$ 
Learning Rate:  $\eta = 0.05$ 
```

Task

1. Forward pass to Z_1 , A_1 , $Z_2 (= \hat{y})$.
 2. Calculate the MSE loss.
 3. Calculate $\partial L / \partial \hat{y}$, then backprop through the network to get $\partial L / \partial W_2$, $\partial L / \partial b_2$, $\partial L / \partial W_1$, $\partial L / \partial b_1$.
 4. Update all parameters.
-

Problem 19: Regression — Comparing MSE, MAE, Huber Loss

Given

```
Prediction:  $\hat{y} = 5$ 
Target:  $y = 8$ 
Huber delta:  $\delta = 2$ 
```

Task

1. Calculate MSE loss: $(\hat{y} - y)^2$.
 2. Calculate MAE loss: $|\hat{y} - y|$.
 3. Determine whether $|\hat{y} - y| > \delta$, then calculate the Huber loss using the correct branch:
 - if $|\hat{y} - y| \leq \delta$: $L = 0.5(\hat{y} - y)^2$
 - if $|\hat{y} - y| > \delta$: $L = \delta(|\hat{y} - y| - 0.5\delta)$
 4. Calculate the derivative $dL/d\hat{y}$ for MSE, MAE, and Huber at this point.
-

Problem 20: Multi-Output Regression

Given

```
Input: x = 1
W = [[2], [1], [-1]]    (3 output neurons, 1 input each)
b = [0.5, -0.5, 0.2]
Activation: identity
Target: y = [3, 1, 0]
Loss: MSE averaged over outputs,  $L = (1/3)\sum(\hat{y}_i - y_i)^2$ 
```

Task

1. Calculate $\hat{y} = [\hat{y}_1, \hat{y}_2, \hat{y}_3]$.
 2. Calculate the total loss L .
 3. Calculate $\partial L / \partial \hat{y}_i$ for each output.
 4. Calculate $\partial L / \partial W$ and $\partial L / \partial b$ (elementwise, since input is scalar).
-

Section 9 — Classification MLPs

Problem 21: Binary Classification — Sigmoid + BCE

Given

```
Input: x = [2, -1]
w = [0.3, 0.4]
b = -0.1
Activation: sigmoid
Target: y = 1
Loss: Binary Cross-Entropy
Learning Rate:  $\eta = 0.2$ 
```

Task

1. Calculate z , then $\hat{y} = \sigma(z)$.
 2. Calculate the BCE loss.
 3. Calculate $\partial L / \partial z$ (use the sigmoid+BCE shortcut $\hat{y} - y$).
 4. Calculate $\partial L / \partial w$ and $\partial L / \partial b$.
 5. Update w and b .
-

Problem 22: Multiclass Classification — Softmax + Cross-Entropy

Given

```
Logits:  $z = [2.0, 1.0, 0.1]$       (3 classes)
True class (one-hot):  $y = [1, 0, 0]$     (i.e., class 0 is correct)
Learning Rate:  $\eta = 0.1$ 
(Assume these logits come directly from a linear layer with input  $x = [1, 1]$ .
To keep this tractable,
you only need to compute the gradient with respect to the logits  $z$ , not all
the way back to weights.)
```

Task

1. Calculate the softmax probabilities $p = [p_0, p_1, p_2]$ using $p_i = e^{z_i} / \sum e^{z_j}$. Use $e = 2.71828$.
 2. Determine the predicted class (argmax of p).
 3. Calculate the categorical cross-entropy loss $L = -\sum y_i \log(p_i)$.
 4. Calculate $\partial L / \partial z$ using the softmax+cross-entropy shortcut: $\partial L / \partial z_i = p_i - y_i$.
-

Problem 23: Sparse Categorical Cross-Entropy — Full Layer With Weights

Given

```
Input:  $x = [1, 1]$ 
Weights:  $W = \begin{bmatrix} 1.0 & 0.5 \\ 0.2 & 0.8 \\ -0.3 & 0.1 \end{bmatrix}$   (3 classes, 2 inputs each)
Biases:  $b = [0.1, 0.0, -0.1]$ 
True class index (sparse label):  $y = 1$     (i.e., class index 1 is correct)
Learning Rate:  $\eta = 0.1$ 
```

Task

1. Calculate logits $z = W \cdot x + b$ (vector of 3).
2. Calculate softmax probabilities p .
3. Calculate the sparse categorical cross-entropy loss $L = -\log(p_y)$ (using only the true class's probability).
4. Calculate $\partial L / \partial z (= p - \text{one_hot}(y))$.
5. Calculate $\partial L / \partial W$ and $\partial L / \partial b$.

6. Update W and b .

Section 10 — Batch Gradient Descent

Problem 24: Batch Gradient Descent — Full Dataset Update

Given

Dataset (scalar regression):

$$x^{(1)} = 1, y^{(1)} = 3$$

$$x^{(2)} = 2, y^{(2)} = 5$$

$$x^{(3)} = 3, y^{(3)} = 7$$

Model: $\hat{y} = wx + b$

Initial weight: $w = 1$

Initial bias: $b = 0$

Loss (per example): $L = (\hat{y} - y)^2$

Learning Rate: $\eta = 0.01$

Task

1. Calculate $\hat{y}$ for all 3 examples.
 2. Calculate the total loss (sum) and average loss (mean) across all 3 examples.
 3. Calculate $\partial L / \partial w$ and $\partial L / \partial b$ for each example individually.
 4. Calculate the **average** gradient across all 3 examples.
 5. Perform **one single update** to w and b using the averaged gradient (this is Batch GD — one update per full pass over the dataset).
-

Section 11 — Stochastic Gradient Descent

Problem 25: SGD — Sequential Updates, One Example At a Time

Given

Same dataset as Problem 24:

$$x^{(1)} = 1, y^{(1)} = 3$$

$$x^{(2)} = 2, y^{(2)} = 5$$

$$x^{(3)} = 3, y^{(3)} = 7$$

Model: $\hat{y} = wx + b$

Initial weight: $w = 1$

Initial bias: $b = 0$

Loss: $L = (\hat{y} - y)^2$

Learning Rate: $\eta = 0.01$

Task

1. Process $x^{(1)}$: compute $\hat{y}$, loss, gradients, and update w, b immediately.
 2. Using the **updated** w, b , process $x^{(2)}$: compute $\hat{y}$, loss, gradients, and update w, b again.
 3. Using the **updated** w, b , process $x^{(3)}$: compute $\hat{y}$, loss, gradients, and update w, b again.
 4. Report the final w, b after this one epoch of SGD, and compare them to the result of Problem 24 (Batch GD). Explain why they differ.
-

Section 12 — Mini-Batch Gradient Descent

Problem 26: Mini-Batch Gradient Descent

Given

Dataset (scalar regression), 4 examples:

$$x^{(1)} = 1, y^{(1)} = 3$$

$$x^{(2)} = 2, y^{(2)} = 5$$

$$x^{(3)} = 3, y^{(3)} = 7$$

$$x^{(4)} = 4, y^{(4)} = 8$$

Mini-batch size: 2 (Batch A = $\{x^{(1)}, x^{(2)}\}$, Batch B = $\{x^{(3)}, x^{(4)}\}$)

Model: $\hat{y} = wx + b$

Initial weight: $w = 1$

Initial bias: $b = 0$

Loss: $L = (\hat{y} - y)^2$

Learning Rate: $\eta = 0.01$

Task

1. For Batch A: calculate $\hat{y}$ and the gradient ($\partial L / \partial w, \partial L / \partial b$) for each of the 2 samples.
 2. Average the two gradients and perform **one update** to w and b .
 3. Using the updated w, b , repeat for Batch B: calculate gradients for each sample, average them, and perform a second update.
 4. Report the final w, b after processing both mini-batches (one epoch).
 5. In your own words (numerically grounded in what you just computed), state how Batch GD, SGD, and Mini-Batch GD differed in *how many updates occurred per epoch* and *how noisy each individual gradient was*.
-

Section 13 — L1 Regularization / Lasso

Problem 27: L1 Regularization — Penalty, Subgradient, Update

Given

```
Weights:  $w = [2, -3, 0.5]$ 
Regularization strength:  $\alpha = 0.1$ 
Data loss gradient ( $\partial L_{\text{data}} / \partial w$ ) =  $[0.4, -0.2, 0.1]$ 
Learning Rate:  $\eta = 0.05$ 
 $L_{\text{total}} = L_{\text{data}} + \alpha \sum |w|$ 
```

Task

1. Calculate the L1 penalty term $\alpha \sum |w_i|$.
 2. Calculate the L1 subgradient for each weight: $\text{sign}(w_i)$ (use $\text{sign}(0)=0$, not relevant here since no $w_i=0$).
 3. Calculate the total gradient $\partial L_{\text{total}} / \partial w = \partial L_{\text{data}} / \partial w + \alpha \cdot \text{sign}(w)$.
 4. Update all weights using gradient descent.
-

Section 14 — L2 Regularization / Ridge

Problem 28: L2 Regularization — Penalty, Gradient, Update

Given

```
Weights: w = [2, -3, 0.5]
Regularization strength:  $\alpha = 0.1$ 
Data loss gradient ( $\partial L_{\text{data}} / \partial w$ ) = [0.4, -0.2, 0.1]
Learning Rate:  $\eta = 0.05$ 
 $L_{\text{total}} = L_{\text{data}} + \alpha \sum w^2$ 
```

Task

1. Calculate the L2 penalty term $\alpha \sum w_i^2$.
 2. Calculate the L2 gradient contribution: $2\alpha \cdot w_i$ for each weight.
 3. Calculate the total gradient $\partial L_{\text{total}} / \partial w = \partial L_{\text{data}} / \partial w + 2\alpha \cdot w$.
 4. Update all weights.
-

Section 15 — Elastic Net

Problem 29: Elastic Net — Combined L1 + L2

Given

```
Weights: w = [2, -3, 0.5]
 $\alpha = 0.1$ 
 $\rho = 0.5$  (mixing ratio between L1 and L2)
Data loss gradient ( $\partial L_{\text{data}} / \partial w$ ) = [0.4, -0.2, 0.1]
Learning Rate:  $\eta = 0.05$ 
 $L_{\text{total}} = L_{\text{data}} + \alpha [\rho \sum |w| + (1-\rho)/2 \sum w^2]$ 
```

Task

1. Calculate the L1 contribution to the gradient: $\alpha \cdot \rho \cdot \text{sign}(w)$.
 2. Calculate the L2 contribution to the gradient: $\alpha \cdot (1-\rho) \cdot w$.
 3. Calculate the combined regularization gradient (sum of the two contributions).
 4. Calculate the total gradient $\partial L_{\text{total}} / \partial w$ and update all weights.
-

Section 16 — PCA

Problem 30: Complete PCA By Hand (2D $\rightarrow$ 1D)

Given

Dataset (4 points, 2 features):

P1 = [2, 0]

P2 = [0, 2]

P3 = [3, 3]

P4 = [-1, -1]

Task

1. Calculate the mean of each feature.
2. Calculate the centered data (subtract the mean from each point).
3. Calculate the 2×2 covariance matrix (use the population formula, dividing by $N=4$, for simplicity).
4. Calculate the eigenvalues of the covariance matrix (solve the characteristic polynomial $\det(C - \lambda I) = 0$).
5. Calculate the eigenvector corresponding to the **larger** eigenvalue (the first principal component), and normalize it to unit length.
6. Calculate the explained variance ratio of this principal component ($\lambda_1 / (\lambda_1 + \lambda_2)$).
7. Project each centered data point onto this principal component (1D projection).
8. Reconstruct the original (approximate) 2D points from the 1D projection and compare to the originals.

Also explain (conceptually, no calculation needed): how running PCA before an MLP affects input dimensionality, training speed, correlated features, and overfitting risk.

Section 17 — Vanishing Gradients

Problem 31: Vanishing Gradient — Sigmoid, 5 Layers

Given

A 5-layer network. At each layer the local derivative of the activation (sigmoid) evaluated at the current pre-activation is:

Layer 1: $\sigma'(z_1) = 0.20$

Layer 2: $\sigma'(z_2) = 0.15$

Layer 3: $\sigma'(z_3) = 0.10$

Layer 4: $\sigma'(z_4) = 0.05$

Layer 5: $\sigma'(z_5) = 0.02$

Each layer's weight (scalar, for simplicity) = 1.0

Gradient arriving at the output layer: $\partial L / \partial a_5 = 1.0$

Task

1. Using Gradient $\approx$ product of derivatives across layers, calculate the gradient that reaches Layer 1 by multiplying $\partial L / \partial a_5$ by each layer's local derivative $\sigma'(z)$ sequentially (Layer 5 $\rightarrow$ Layer 1).
 2. Report the gradient magnitude at each layer as you move backward, and observe how quickly it shrinks.
 3. State, numerically, what fraction of the original gradient (1.0) remains by the time it reaches Layer 1.
-

Section 18 — Exploding Gradients

Problem 32: Exploding Gradient — Large Weights, 5 Layers

Given

A 5-layer network. At each layer:

weight = 3.0

local activation derivative ≈ 1.0 (e.g., ReLU active region)

Gradient arriving at the output layer: $\partial L / \partial a_5 = 1.0$

Task

1. Calculate the gradient that reaches each layer going backward, by multiplying by (weight $\times$ local derivative) = 3.0 at each step.
2. Report the gradient magnitude at each of the 5 layers.

3. Compare the growth rate here to the shrink rate in Problem 31, and state the general rule connecting the magnitude of (weight $\times$ derivative) at each layer to whether gradients vanish or explode.
-

Section 19 — Weight Initialization (Poor Initialization)

Problem 33: All-Zero Initialization — Symmetry Breaking Failure

Given

```
A tiny network: 1 input, 1 hidden layer with 2 neurons, 1 output neuron.  
All weights initialized to 0. All biases initialized to 0.  
Input:  $x = 1.5$   
Activation: ReLU (hidden), identity (output)
```

Task

1. Calculate Z_1 (both hidden neurons) and A_1 .
 2. Calculate the output.
 3. Calculate $\partial L / \partial W_1$ for both hidden neurons symbolically/numerically (they will be identical) and explain — using the numbers you just computed — why both hidden neurons will always receive the exact same gradient and therefore never differentiate from each other during training.
-

Problem 34: Very Large vs Very Small Random Initialization

Given

```
Case A (large init):  $W = [10, -8, 12]$ ,  $x = [0.5, 0.5, 0.5]$ ,  $b = 0$ , activation = sigmoid  
Case B (small init):  $W = [0.001, -0.002, 0.0015]$ ,  $x = [0.5, 0.5, 0.5]$ ,  $b = 0$ , activation = sigmoid
```

Task

1. For Case A: calculate z , then $\sigma(z)$, then $\sigma'(z)$. Comment on how close $\sigma'(z)$ is to zero (saturation).
2. For Case B: calculate z , then $\sigma(z)$, then $\sigma'(z)$. Comment on how close $\sigma(z)$ is to 0.5 and what that implies for how much signal is being distinguished.

3. Explain, using these two numerical results, why both extremes hurt training (one saturates and kills gradients, the other produces almost no useful signal).
-

Section 20 — Glorot/Xavier Initialization

Problem 35: Xavier Initialization — Variance Calculation

Given

```
n_inputs = 6
n_outputs = 4
```

Task

1. Calculate the Xavier variance: $\sigma^2 = 2/(n_inputs + n_outputs)$.
 2. Calculate the standard deviation $\sigma = \sqrt{\sigma^2}$.
 3. State the range for the corresponding uniform-distribution version: limit = $\sqrt{6/(n_inputs+n_outputs)}$, so weights would be drawn from $[-limit, +limit]$.
 4. Give 3 example weight values that would be plausible draws within one standard deviation of 0 given this σ .
-

Section 21 — He Initialization

Problem 36: He Initialization — Variance Calculation

Given

```
n_inputs = 8
(Used for a layer followed by ReLU.)
```

Task

1. Calculate the He variance: $\sigma^2 = 2/n_inputs$.
2. Calculate the standard deviation σ .
3. Give 3 example weight values that would be plausible draws within one standard deviation of 0.
4. Explain numerically why He initialization uses double the variance of a naive $1/n_inputs$ scheme — connect it to the fact that ReLU zeroes out roughly half of its inputs.

Section 22 — Nonsaturating Activation Functions (Comparison)

Problem 37: Compare All 6 Activations at the Same Input

Given

```
z = -1.5      (same input value for every activation)
Sigmoid, Tanh, ReLU, Leaky ReLU ( $\alpha=0.01$ ), ELU ( $\alpha=1.0$ ), SELU ( $\lambda=1.0507$ ,  $\alpha=1.6733$ )
Use e = 2.71828
```

Task

1. Calculate the forward output of all 6 activations at $z = -1.5$.
2. Calculate the derivative of all 6 activations at $z = -1.5$.
3. Rank the 6 activations by the magnitude of their derivative at this point (largest to smallest).
4. Based on your numbers, state which activation(s) risk vanishing gradients at this input and which do not, and in what situations (deep networks, self-normalizing networks, need for zero-centered outputs, etc.) each would be preferred.

Section 23 — Batch Normalization

Problem 38: Batch Normalization — Forward Pass

Given

```
Mini-batch activations: x = [1, 2, 3, 4]
 $\gamma = 2$ 
 $\beta = 1$ 
 $\epsilon = 0.001$ 
```

Task

1. Calculate the mini-batch mean μ_B .
2. Calculate the mini-batch variance σ_B^2 (population formula, divide by $N=4$).
3. Calculate the normalized activations $\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma_B^2 + \epsilon)}$ for all 4 values.
4. Calculate the scale-and-shift output $y_i = \gamma \hat{x}_i + \beta$ for all 4 values.

Where it's applied: state whether BatchNorm is applied before or after the linear layer's weighted sum, and before or after the activation function, in the standard convention (Dense → BatchNorm → Activation).

Problem 39: Batch Normalization — Simplified Backward Pass

Given

```
Continue from Problem 38's values: x = [1,2,3,4],  $\mu_B$ ,  $\sigma_B^2$ ,  $\hat{x}$ ,  $\gamma=2$ ,  $\beta=1$ ,  
 $\epsilon=0.001$   
Upstream gradient:  $\partial L / \partial y = [0.1, -0.2, 0.05, 0.15]$  (one value per batch  
element)
```

Task

1. Calculate $\partial L / \partial \gamma = \sum_i (\partial L / \partial y_i \cdot \hat{x}_i)$.
 2. Calculate $\partial L / \partial \beta = \sum_i \partial L / \partial y_i$.
 3. Calculate $\partial L / \partial \hat{x}_i = \partial L / \partial y_i \cdot \gamma$ for each i (you may stop here — full $\partial L / \partial x$ through the mean/variance terms is optional/advanced, but attempt it if you want the complete picture using the standard BatchNorm backward formula).
-

Section 24 — Gradient Clipping

Problem 40: Value Clipping

Given

```
gradient = [10, -8, 0.5]  
clip_value = 1
```

Task: Apply value clipping — clip every component to the range $[-\text{clip_value}, +\text{clip_value}]$ — and report the clipped gradient.

Problem 41: Norm Clipping

Given

```
gradient g = [3, 4]
max_norm threshold = 2
```

Task

1. Calculate the L2 norm $\|g\| = \sqrt{g_1^2 + g_2^2}$.
2. Determine whether $\|g\|$ exceeds the threshold.
3. If it does, calculate $g_{\text{clipped}} = g \times \text{threshold} / \|g\|$.
4. Verify that $\|g_{\text{clipped}}\|$ equals the threshold.

Explain: when is gradient clipping most useful (which architectures/situations), and how does norm clipping differ from value clipping in terms of preserving the gradient's *direction*?

Section 25 — Momentum Optimization

Problem 42: Momentum — 3 Consecutive Steps

Given

```
Initial parameter:  $\theta_0 = 5$ 
Initial velocity:  $v_0 = 0$ 
Gradients at each step (pretend these are given/observed, not recomputed):  $g_1 = 2$ ,  $g_2 = 1.5$ ,  $g_3 = 1$ 
Learning Rate:  $\eta = 0.1$ 
Momentum coefficient:  $\beta = 0.9$ 
Update rule:
   $v_t = \beta v_{t-1} + g_t$ 
   $\theta_t = \theta_{t-1} - \eta v_t$ 
```

Task: Calculate v_1, θ_1 , then v_2, θ_2 , then v_3, θ_3 — three full consecutive momentum update steps, in order.

Section 26 — Nesterov Accelerated Gradient

Problem 43: NAG vs Momentum — Comparison at Step 2

Given

```
θ1 = 4.8    (parameter after step 1, both methods started identically)
v1 = 2.0    (velocity after step 1, both methods)
Learning Rate: η = 0.1
Momentum coefficient: β = 0.9
Gradient function: g(θ) = 0.5θ (a simple linear gradient function so you can
evaluate g at any θ)
```

Task — Plain Momentum:

1. Calculate $g(\theta_1)$ using the gradient function.
2. Calculate $v_2 = \beta \cdot v_1 + g(\theta_1)$.
3. Calculate $\theta_2 = \theta_1 - \eta \cdot v_2$.

Task — Nesterov Accelerated Gradient: 4. Calculate the look-ahead position: $\theta_{\text{lookahead}} = \theta_1 - \eta \cdot \beta \cdot v_1$. 5. Calculate the gradient **at the look-ahead position**: $g(\theta_{\text{lookahead}})$. 6. Calculate $v_{2_NAG} = \beta \cdot v_1 + g(\theta_{\text{lookahead}})$. 7. Calculate $\theta_{2_NAG} = \theta_1 - \eta \cdot v_{2_NAG}$.

Compare: report θ_2 (momentum) vs θ_{2_NAG} side by side and explain, using your two different gradient evaluations ($g(\theta_1)$ vs $g(\theta_{\text{lookahead}})$), why NAG "corrects" the update before committing to it.

Section 27 — AdaGrad

Problem 44: AdaGrad — 3 Steps, Adaptive Learning Rate

Given

```
Initial parameter: θ0 = 3
Initial accumulator: s0 = 0
Gradients: g1 = 1, g2 = 0.8, g3 = 0.6
Learning Rate: η = 0.5
ε = 1e-8
Update rule:
    s_t = s_{t-1} + g_t^2
    θ_t = θ_{t-1} - (η / (√s_t + ε)) · g_t
```

Task: Calculate s_1, θ_1 , then s_2, θ_2 , then s_3, θ_3 . At each step, report the **effective learning rate** $\eta / (\sqrt{s_t} + \epsilon)$ and observe how it shrinks as s_t accumulates.

Section 28 — RMSProp

Problem 45: RMSProp — 3 Steps

Given

```
Initial parameter:  $\theta_0 = 3$ 
Initial accumulator:  $s_0 = 0$ 
Gradients:  $g_1 = 1, g_2 = 0.8, g_3 = 0.6$ 
Learning Rate:  $\eta = 0.1$ 
Decay rate:  $\beta = 0.9$ 
 $\epsilon = 1e-8$ 
Update rule:
 $s_t = \beta s_{t-1} + (1-\beta) g_t^2$ 
 $\theta_t = \theta_{t-1} - (\eta / (\sqrt{s_t} + \epsilon)) \cdot g_t$ 
```

Task: Calculate s_1, θ_1 , then s_2, θ_2 , then s_3, θ_3 . Report the effective learning rate at each step.

Compare: using your numbers from Problem 44 (AdaGrad) and this problem (RMSProp), explain why RMSProp's accumulator does not monotonically grow the way AdaGrad's does, and what practical benefit that provides for training over many steps.

Section 29 — Adam

Problem 46: Adam — 2 Full Optimization Steps

Given

```
Initial parameter:  $\theta_0 = 2$ 
Initial first moment:  $m_0 = 0$ 
Initial second moment:  $v_0 = 0$ 
Gradients:  $g_1 = 0.6, g_2 = 0.5$ 
Learning Rate:  $\eta = 0.1$ 
 $\beta_1 = 0.9$ 
 $\beta_2 = 0.999$ 
 $\epsilon = 1e-8$ 
Update rules:
 $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ 
 $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ 
 $\hat{m}_t = m_t / (1 - \beta_1^t)$ 
 $\hat{v}_t = v_t / (1 - \beta_2^t)$ 
 $\theta_t = \theta_{t-1} - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ 
```

Task

1. **Step 1 (t=1):** calculate m_1 , v_1 , then bias-corrected $\hat{m}_1$ and $\hat{v}_1$ (remember $\beta_1^1 = 0.9$ and $\beta_2^1 = 0.999$), then θ_1 .
 2. **Step 2 (t=2):** using m_1 , v_1 , and g_2 , calculate m_2 , v_2 , bias-corrected $\hat{m}_2$ and $\hat{v}_2$ (using $\beta_1^2 = 0.81$ and $\beta_2^2 = 0.998001$), then θ_2 .
 3. Report $\theta_0, \theta_1, \theta_2$ in sequence.
-

Section 30 — Nadam

Problem 47: Nadam — Incorporating Nesterov Momentum into Adam

Given

```
Initial parameter:  $\theta_0 = 2$ 
Initial first moment:  $m_0 = 0$ 
Initial second moment:  $v_0 = 0$ 
Gradient at t=1:  $g_1 = 0.6$ 
Learning Rate:  $\eta = 0.1$ 
 $\beta_1 = 0.9$ 
 $\beta_2 = 0.999$ 
 $\epsilon = 1e-8$ 
Nadam update (single step, t=1):
   $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ 
   $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ 
   $\hat{m}_t = m_t / (1 - \beta_1^t)$ 
   $\hat{v}_t = v_t / (1 - \beta_2^t)$ 
  Nesterov-style numerator:  $m\_nadam = \beta_1 \cdot \hat{m}_t + ((1-\beta_1)/(1-\beta_1^t)) \cdot g_t$ 
   $\theta_t = \theta_{t-1} - \eta \cdot m\_nadam / (\sqrt{\hat{v}_t} + \epsilon)$ 
```

Task

1. Calculate m_1 and v_1 .
2. Calculate the bias-corrected $\hat{m}_1$ and $\hat{v}_1$.
3. Calculate the Nadam Nesterov-style numerator m_nadam .
4. Calculate θ_1 .

Compare: using the $\hat{m}_1$ value here and the equivalent $\hat{m}_1$ you computed in Problem 46 (Adam) — they should match since inputs are identical at $t=1$ — explain, in terms of the formula, exactly where Nadam's update to θ_1 diverges from plain Adam's update to θ_1 (i.e., what extra term is injected).

Section 31 — AdaMax

Problem 48: AdaMax — Infinity Norm Update, 2 Steps

Given

```
Initial parameter:  $\theta_0 = 2$ 
Initial first moment:  $m_0 = 0$ 
Initial infinity-norm accumulator:  $u_0 = 0$ 
Gradients:  $g_1 = 0.6$ ,  $g_2 = -0.9$ 
Learning Rate:  $\eta = 0.1$ 
 $\beta_1 = 0.9$ 
 $\beta_2 = 0.999$ 
Update rules:
   $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ 
   $u_t = \max(\beta_2 u_{t-1}, |g_t|)$ 
   $\hat{m}_t = m_t / (1 - \beta_1^t)$ 
   $\theta_t = \theta_{t-1} - \eta \hat{m}_t / u_t$ 
```

Task

1. **Step 1:** calculate m_1 , u_1 (compare $\beta_2 \cdot u_0 = 0$ vs $|g_1| = 0.6$ and take the max), $\hat{m}_1$ (using $\beta_1^1=0.9$), then θ_1 .
2. **Step 2:** calculate m_2 , u_2 (compare $\beta_2 \cdot u_1$ vs $|g_2| = 0.9$ and take the max), $\hat{m}_2$ (using $\beta_1^2=0.81$), then θ_2 .

Compare: contrast the u_t update rule here (a running max) against Adam's v_t update rule (an exponential moving average of squared gradients) — using the numbers you just computed for u_1 and u_2 , explain why AdaMax's accumulator can never shrink.

Section 32 — Integrated Deep Learning Problems

Problem 49: Integrated Problem 1 — Dense → BatchNorm → ReLU → Dense → Sigmoid → BCE → L2 → Adam

Given

Input: $x = [1, 2]$

Layer 1 (Dense): $W1 = \begin{bmatrix} 0.2 & 0.1 \\ 0.4 & -0.3 \end{bmatrix}$
 $b1 = [0.0, 0.1]$

BatchNorm (applied to the 2 pre-activation values $Z1$ as if they were a "batch" of 2 scalars, purely for hand-calculation practice):
 $\gamma = [1, 1], \beta_{bn} = [0, 0], \epsilon = 0.001$

Activation 1: ReLU

Layer 2 (Dense, output): $W2 = [0.5, -0.4]$
 $b2 = 0.05$

Activation 2: sigmoid

Target: $y = 1$

Loss: Binary Cross-Entropy

L2 regularization strength: $\alpha = 0.01$ (applied to $W1$ and $W2$)

Optimizer: Adam with $\eta=0.1$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$, starting at $t=1$, all first/second moment accumulators start at 0.

Task

1. Forward: calculate $Z1 = W1 \cdot x + b1$.
 2. Apply BatchNorm to the 2 values of $Z1$ (treat them as a batch of size 2: compute mean, variance, normalize, scale/shift) to get $Z1_{bn}$.
 3. Apply ReLU to get $A1$.
 4. Calculate $Z2 = W2 \cdot A1 + b2$, then $\hat{y} = \sigma(Z2)$.
 5. Calculate the BCE data loss, the L2 penalty ($\alpha(\Sigma W1^2 + \Sigma W2^2)$), and the total loss.
 6. Backward: calculate $\partial L / \partial Z2$ (sigmoid+BCE shortcut), $\partial L / \partial W2$, $\partial L / \partial b2$ (data term only).
 7. Add the L2 gradient contribution ($2\alpha W2$) to $\partial L / \partial W2$.
 8. Continue backprop through ReLU and (approximately, using $\partial \hat{x} / \partial x \approx 1 / \sqrt{(\sigma_B^2 + \epsilon)}$ as a simplification) through BatchNorm to reach $\partial L / \partial Z1$, then $\partial L / \partial W1$, $\partial L / \partial b1$, and add the L2 term ($2\alpha W1$).
 9. Apply **one Adam update step** ($t=1$) to $W1$, $b1$, $W2$, $b2$ using the total gradients from steps 7-8.
-

Problem 50: Integrated Problem 2 — PCA → MLP → BatchNorm → ELU → Dense → Softmax → CE → Clipping → RMSProp

Given

Raw dataset (2 features, for the PCA step):

$P1 = [4, 2]$

$P2 = [2, 0]$

$P3 = [0, -2]$

(use these 3 points only for the PCA step)

After PCA, use the resulting 1D projection of P1 as the network's input for the rest of this problem

(i.e., compute the PC1 projection of P1 first, then treat that single number as scalar input x for the MLP below).

MLP Hidden Layer: $w1 = [0.5, -0.2]$ (2 hidden neurons, 1 input each – since x is scalar after PCA)

$b1 = [0.1, -0.1]$

BatchNorm (on the 2 hidden pre-activations, treated as a batch of 2 for hand practice):

$\gamma = [1, 1], \beta_{bn} = [0, 0], \epsilon = 0.001$

Activation: ELU ($\alpha = 1.0$)

Output Layer (3-class softmax): $W2 = \begin{bmatrix} 0.3 & 0.1 \\ 0.2 & -0.4 \\ -0.1 & 0.5 \end{bmatrix}$

$b2 = [0, 0, 0]$

True class: $y = 0$

Loss: Categorical Cross-Entropy

Gradient clipping: clip by norm, $\text{max_norm} = 1$ (apply to the final $\partial L / \partial W2$ matrix, treated as one flattened vector)

Optimizer: RMSProp, $\eta = 0.1, \beta = 0.9, \epsilon = 1e-8$, accumulators start at 0

Task

1. PCA: calculate the mean of the 3 points, center them, compute the covariance matrix, find the dominant eigenvector (PC1), and project P1 onto it to get scalar x .
2. Forward through the MLP hidden layer to $Z1$, apply BatchNorm (batch of 2) to get $Z1_{bn}$, apply ELU to get $A1$.

3. Forward through the output layer to get logits Z2 (3 values), then softmax probabilities p, then the categorical cross-entropy loss.
 4. Backward: calculate $\partial L / \partial Z2$ (softmax+CE shortcut: $p - \text{one_hot}(y)$), then $\partial L / \partial W2$ and $\partial L / \partial b2$.
 5. Apply gradient clipping (by norm, threshold=1) to the flattened $\partial L / \partial W2$ gradient — calculate the norm first, then clip if needed.
 6. Continue backprop through ELU, BatchNorm (using the same $1/\sqrt{(\sigma_B^2 + \epsilon)}$ approximation as in Problem 49), to get $\partial L / \partial w1$ and $\partial L / \partial b1$.
 7. Apply **one RMSProp update step** to W2 (using the clipped gradient) and to w1, b1 (unclipped).
-

Problem 51: Integrated Problem 3 — SELU MLP + Proper Init + BatchNorm + L1 + Mini-Batch + Momentum

Given

Mini-batch of 2 examples:

$$x^{(1)} = [1, -1], y^{(1)} = 1$$

$$x^{(2)} = [0.5, 0.5], y^{(2)} = 0$$

Layer (SELU-activated): $W = \begin{bmatrix} 0.3 & -0.2 \\ 0.1 & 0.4 \end{bmatrix}$ (2 hidden neurons)

$$b = [0, 0]$$

(Assume these were drawn appropriately under a LeCun-normal / SELU-style

initialization scheme — verify this is plausible:

$n_{\text{inputs}}=2$, so the target

variance for SELU init is $1/n_{\text{inputs}} = 0.5$, target

std ≈ 0.707 . Check whether

the given weight magnitudes are broadly consistent

with that std.)

SELU: $\lambda = 1.0507$, $\alpha = 1.6733$

Output (linear, single neuron, regression-style for simplicity): $w_{\text{out}} = [0.6, -0.5]$, $b_{\text{out}} = 0$

Target for output: treat $y^{(1)}$, $y^{(2)}$ above as regression targets directly.

Loss: MSE, averaged over the mini-batch

L1 regularization: $\alpha_{\text{L1}} = 0.01$ applied to W (hidden layer) only

Learning Rate: $\eta = 0.05$

Momentum: $\beta = 0.9$, $v_0 = 0$ (for every parameter)

Task

1. Verify the initialization plausibility as described above (one short calculation).
 2. Forward pass for $x^{(1)}$: $Z, A = \text{SELU}(Z)$, output $\hat{y}^{(1)}$. Same for $x^{(2)}$.
 3. Calculate the MSE loss for each example and the batch-averaged loss.
 4. Backprop for each example to get $\partial L / \partial w_{\text{out}}, \partial L / \partial b_{\text{out}}, \partial L / \partial W, \partial L / \partial b$ — then average the two examples' gradients (mini-batch averaging).
 5. Add the L1 subgradient term ($\alpha_{\text{L1}} \cdot \text{sign}(W)$) to the averaged $\partial L / \partial W$.
 6. Apply **one momentum update step** to all parameters ($w_{\text{out}}, b_{\text{out}}, W, b$) using the averaged (and, for W , L1-augmented) gradients.
-

Problem 52: Integrated Problem 4 — Deep MLP, Vanishing/Exploding Analysis, Clipping, He Init, Activation & Optimizer Comparison

Given

A 4-layer deep network (scalars only, for tractable hand analysis), all with weight = w and local activation derivative $f'(z)$ as listed:

Configuration A ("small weights", using ReLU): $w=0.4$ at every layer, $f'(z)=1$ at every layer

(since ReLU is active) for layers 1-4.

Configuration B ("large weights"): $w=2.5$ at every layer, $f'(z)=1$ at every layer for layers 1-4.

Gradient arriving at the output (layer 4): $\partial L / \partial a_4 = 1.0$

Gradient clipping threshold (norm-based, applied to the single scalar gradient once it's

computed for layer 1 in Configuration B): $\max_norm = 5$

Separately, for the initialization check:

$n_inputs = 4$ for a layer about to use ReLU.

Also, for the optimizer comparison at the end, use this single (parameter, gradient) pair for

ALL THREE optimizers below so the comparison is apples-to-apples:

$\theta_0 = 1$, $g = 0.4$ (single, fixed gradient value – used identically in Adam, Nadam, and AdaMax,

as if it were both g_1 and g_2 for a fair 2-step comparison)

$\eta=0.1$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$, all moment accumulators start at 0

Task

Part A — Vanishing/Exploding analysis:

1. For Configuration A, calculate the gradient reaching layer 1 by multiplying $\partial L / \partial a_4$ by $(w \times f'(z))$ four times (once per layer, moving backward from layer 4 to layer 1).
2. For Configuration B, do the same multiplication four times.
3. Compare the two resulting gradient magnitudes at layer 1 and classify each configuration as exhibiting vanishing or exploding gradient behavior.

Part B — Gradient Clipping: 4. Apply norm clipping ($\max_norm=5$) to the layer-1 gradient you computed for Configuration B. Report the clipped value.

Part C — He Initialization: 5. Calculate the He initialization variance and standard deviation for $n_inputs=4$, and explain briefly (referencing your Part A numbers) why this

initialization scheme, applied consistently, helps avoid the exploding/vanishing behavior you just observed with fixed $w=2.5$ or $w=0.4$.

Part D — ReLU vs ELU comparison: 6. At $z = -0.3$, calculate $\text{ReLU}(z)$, $\text{ReLU}'(z)$, $\text{ELU}(z)$ ($\alpha=1$), and $\text{ELU}'(z)$. State which one has a non-zero gradient here and why that matters for a "dead neuron" at this input.

Part E — Adam vs Nadam vs AdaMax comparison: 7. Using the shared (θ_0, g) values given, calculate θ_1 (i.e., a single first update step, $t=1$) under: (a) Adam, (b) Nadam, (c) AdaMax — reusing the exact update formulas from Sections 29–31. 8. Rank the three resulting θ_1 values and, using the numbers, describe the direction/magnitude difference each optimizer's mechanism (bias correction, Nesterov lookahead, infinity norm) produced from the *identical* starting gradient.

Summary — Full Problem Index (Easiest → Hardest)

#	Problem	Topic
1-2	Scalar neuron	Forward pass, full update
3-4	Vector neuron	Forward pass, gradients & update
5-6	Perceptron	Single update, multi-point classification
7-13	Activation functions	ReLU, Leaky ReLU, ELU, SELU, Sigmoid, Tanh
14	Single-layer forward prop	
15	MLP forward prop	
16-17	Backpropagation	1 hidden layer, 2 hidden layers
18-20	Regression MLPs	MSE update, MSE/MAE/Huber comparison, multi-output
21-23	Classification MLPs	Binary (sigmoid+BCE), softmax+CE, sparse CE with weights
24	Batch Gradient Descent	
25	SGD	
26	Mini-Batch GD	
27	L1 / Lasso	
28	L2 / Ridge	

#	Problem	Topic
29	Elastic Net	
30	PCA	Full manual PCA, 2D→1D
31	Vanishing gradients	
32	Exploding gradients	
33-34	Weight init (poor)	Zero init, large/small random
35	Xavier/Glorot init	
36	He init	
37	Nonsaturating activations	Full 6-way comparison
38-39	Batch Normalization	Forward, backward
40-41	Gradient clipping	Value clipping, norm clipping
42	Momentum	3 steps
43	Nesterov (NAG)	vs Momentum comparison
44	AdaGrad	3 steps
45	RMSProp	3 steps, vs AdaGrad
46	Adam	2 full steps with bias correction
47	Nadam	vs Adam comparison
48	AdaMax	2 steps, vs Adam
49-52	Integrated problems	Full pipelines combining 6-8 techniques each

Remember: I will not solve any of these until you ask. Just say "Solve Problem X step by step" (or reference a range, e.g. "Solve Problems 1-6") and I'll walk through every calculation with no arithmetic skipped.